

ROSIE Ontology

Document Name: ROSIE ontology

Release: 2.3

Creation data: 24/05/2024

Update data: 01/09/2026 (include examples)

04/02/2026 (include references ontologies)

30/06/2026 (include comparison among ontology concepts)

Authors: Ana Patrícia F. Magalhães Mascarenhas

Joao Fabro

Marco Aurélio Wehrmeister

Document purpose: This document contains the specification of ROSIE - Robotics Ontology for Serving Individuals in Everyday tasks. The text is organized as follows: Section 1 presents the ontology main goal and applied domain and Section 2 describes the specification methodology. Section 3 lists the questions that guided the ontology specification, and Section 4 lists the knowledge data source. Section 5 presents the ontology dictionary of concepts, comprising each concept, its definition, and examples. In addition, the annexes contain a brief description of the set of concepts reused from other domain ontologies and upper ontologies.

1. Ontology Main Goal and domain

The main goal of ROSIE is to define a *well-defined knowledge base* for the service robot domain, especially for the RoboCup@Home context, which could:

- Prevent unambiguous communication.
- Support data integration and information exchange
- Support tools and framework implementation, e.g., tools for task planning and code generation.

2. Methodology

In order to define this ontology we integrate the development method proposed by IEEE (2015) and the guidelines proposed by (Vital and Café, 2011).

The work starts by defining the ontology's main goal and domain identification, as well as the specification of sources to acquire knowledge. Current ontologies should be

considered due to its relevance for concepts reuse and specialization and to enable compatibility among ROSIE and other ontologies.

The next step consists of defining the ontology's concepts and properties, and organizing them using an intermediate representation. In this project, we specify a *dictionary of concepts* in natural language and represent it in a metamodel.

Based on the dictionary of concepts, the ontology classes and hierarchy are specified as well as their axioms, object property and data property. Then, instances are defined for each class. In this project we used the Protégé tool for implementation.

Ontology validation consists of three parts: (i) query execution in Protégé simulating each ontology question previously defined in the first step of this methodology. (ii) submit a survey to the RoboCup@home Brazil team to evaluate the dictionary of concepts according to clarity, legibility, coherence, and completeness; (iii) adopt the ontology as a specification instrument by some teams of RoboCup@home competition.

3. Service@Home Main Questions

- I. What are the parts of a robot?
- II. What are the capabilities of a robot?
- III. In which scenario can a service robot act?
- IV. What is a scenario composition in (sub-)areas?
- V. Who can control a robot?
- VI. What are the privileges of a given person?
- VII. What objects can a robot recognize?
- VIII. Where is an object in the environment?
- IX. What are the capabilities required to perform a task?

4. Knowledge source

The ontology specification is based on the following data sources: Current ontologies, the RoboCup@Home rule book and @Home community experience.

5. Reference ontologies

This project reuses / specializes concepts from current standards / ontologies. This is important to enable compatibility among specifications. The following ontologies are used as reference:

- Descriptive ontology for linguistic and cognitive engineering (DOLCE) ultra light (DUL) ;
- Suggested Upper Merged Ontology (SUMO) - is a top-level ontology that defines basic ontological categories across all domains;
- Core ontology for robotics and automation (CORA) – defines the main, most general concepts, relations, and axioms of robotics and automation;

- Autonomous Robotics (AuR) - extend the CORA ontology to represent more specific concepts and axioms that are commonly used in autonomous robotics.

6. ROSIE Dictionary of Concepts

ROSIE organizes the concepts into three categories:

- Concepts related to the robot's structure and behavior.
- Concepts related to the environment definition, where the robot will be inserted
- Concepts related to task definitions that can be performed when using the robot in an environment.

Through the dictionary, concepts are highlighted in bold. For each concept, the dictionary presents its definition and examples.

Reused/extended concepts from other ontologies bring the reference in parenthesis, e.g., (CORA) or (ISO8373:2021).

1.1 Concepts related to the robot structure and behavior

Figure 1 summarizes in a metamodel the concepts specified for defining the robot structure. In general, a **ServiceRobot** is composed of various **ServiceRobotParts**, which include both software and hardware components. These parts work together to enable the robot's set of **Capabilities**, which are implemented through **ServiceRobotBehavior**.

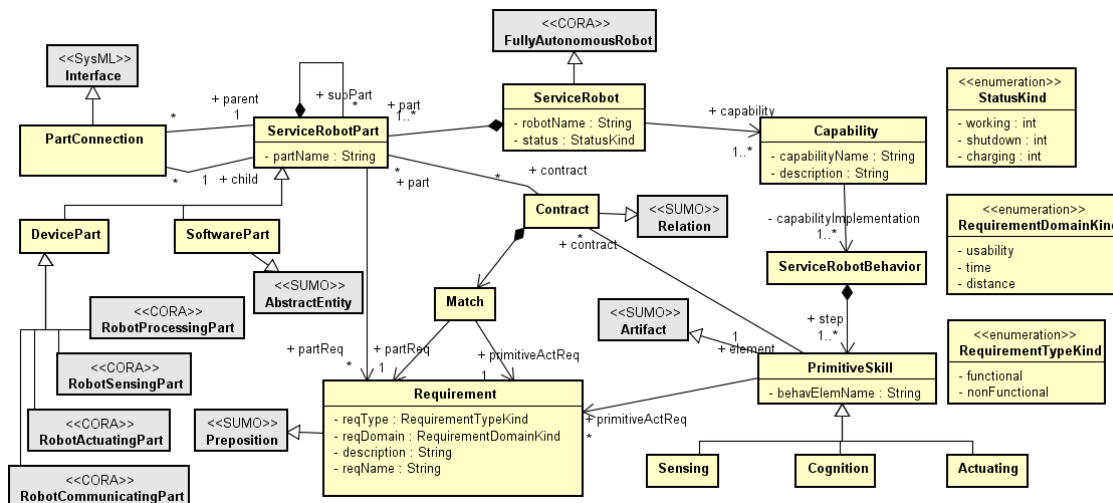


Figure 1: Robot definition metamodel.

ServiceRobot: is a **fully autonomous Robot** (from CORA) for personal or professional use that performs useful tasks for humans or equipment (ISO8373:2021). It may involve **human interaction** (ISO8373:2021) but does not allow **human intervention**.

Human Interaction (ISO8373:2021): “information and action exchanges between human and robot to perform a task by means of a user interface. Exchanges through vocal, visual and tactile means”.

Human intervention: external (remote or in-person) human control.

Personal use: includes tasks related to personal assistance such as cooking, cleaning, carrying things, and handling or serving of items.

Professional use includes tasks related to personal assistance in a non-structured environment, such as, serving dinner in a restaurant, welcoming a hotel guest.

A *ServiceRobot* has a name and a status, indicating whenever it is working, shutdown or charging.

* *ServiceRobot* does NOT include industrial robots, medical robots, or teleoperated robots. In some situations, a robot may assume more than one role, e.g., a hybrid *ServiceRobot* and teleoperated robot. Although this is possible, it is outside this ontology's scope.

* This ontology does not consider Robot Group (CORA). Cooperation among robots to achieve a goal may be considered in future releases.

ServiceRobotPart: represents the parts that compose a *ServiceRobot*. A *ServiceRobotPart* can be composed of another *ServiceRobotPart* (named subpart). A *ServiceRobot* may have hardware parts (*DevicePart*) or software parts (*SoftwarePart*). A part can be connected to other parts through the definition of *PartConnection*.

SoftwarePart: is an abstract entity (from SUMO) that represents the software components of a *ServiceRobot*, e.g., software functions, libraries, and applications.

DevicePart: represents the hardware parts (*RobotPart* in CORA) of a service robot, i.e. a *RobotProcessingPart* (from CORA), a *RobotSensingPart* (from CORA), a *RobotActuatingPart* (from CORA) or a *RobotCommunicatingPart* (from CORA). A *DevicePart* can be physically connected to the robot or logically connected (e.g. an external computer used during a task).

Examples of processing parts for the AtHome domain include computer processors and microcontrollers.

Examples of sensing parts for the AtHome domain include a camera, LIDAR, Kinect, and microphone.

Examples of actuating parts for the AtHome domain include arms, speakers, screens, and wheels.

Examples of communicating part for the AtHome domain include: hub.

PartConnection: is a connection (from SysML), physical or logical, between two *RobotPart*. For example, the connection of a *RobotSensingPart* (e.g. a camera) with a

RobotPart (e.g. chassis), e.g. in Gazebo simulator called *Joint*; the connection of a *SoftwarePart* (e.g. a library) with a *RobotSensingPart* (e.g. camera), in Gazebo Simulator called *Plugin*.

Capability: is the combination of behaviors, robot parts, and their organization to deliver a specified outcome. A capability specifies WHAT the robot can deliver without defining HOW to do it.

ServiceRobotBehavior: represents a capability realization according to a specific implementation strategy. It orchestrates (plans) the set of *PrimitiveSkills*, necessary for a capability implementation. The orchestration consists of the coordination of *PrimitiveSkills*, i.e., involves the definition of a workflow or pipeline, with the *PrimitiveSkills* needed to perform the behavior. *PrimitiveSkills* can be orchestrated in parallel or sequentially using, for example, behavior tree or UML state machine.

PrimitiveSkill: consists of using service robot parts to produce a specific result. A *PrimitiveSkill* is a behavior ATOMIC STAGE, i.e., it CAN NOT be composed by other *PrimitiveSkill*.

Sensing: is a *PrimitiveSkill* related to sensing and capturing data from the environment. Examples are: see (capture a video) and hear (capture sounds).

Cognition: is a *PrimitiveSkill* related to data processing. Examples: Robot training and face recognition.

Actuating: is a *PrimitiveSkill* related to act and modify the environment. Examples: speak.

Contract: establishes an agreement between a *PrimitiveSkills* and a *ServiceRobotPart*. It links a specific *PrimitiveSkill* to the necessary parts for its implementation, i.e., there should be a *Contract* relating the *PrimiviteSkill* to the *ServiceRobotParts* required so that the *PrimitiveSkill* can be implemented/executed by a *ServiceRobot* that has that *ServiceRobotParts*.

A *PrimitiveSkill* may require n parts. For each part, a contract is defined. In a *Contract*, the *PrimitiveSkill* and the *ServiceRobotPart's* non-functional requirements must be compatible* (*Match*). Examples: (i) a *SoftwarePart* that executes in 0.5s is compatible with a *PrimitiveSkill* that requires execution in a maximum of 1s; (ii) A camera (*RobotSensingPart*) that provides an image at 10 frames/s is compatible with a *PrimitiveSkill* that requires an image at least 5 frames/s; (iii) a *DevicePart* that uses a maximum of 50mA is compatible with a *PrimitiveSkill* that requires up to 70mA.

Match: indicates the *PrimitiveSkill* requirement that must be met by the *ServiceRobotPart* requirement.

*The current version of the ServiceRobots ontology does not intend to verify non-functional requirements compatibility automatically, i.e., a human should do this work during behavior element specification.

Requirement: represents a proposition (from SUMO) that must be fulfilled. It has a description of its purpose, a type (reqType), belongs to a domain (reqDomain), and is specialized in *PartRequirement* and *CapabilityRequirement*.

RequirementTypeKind: indicates whenever it refers to a functional or non-functional requirement.

RequirementDomainKind: represents the requirement domain, e.g., usability, time (execution time), or distance (minimal distance, e.g., to see or hear).

Additional examples of the mentioned concepts can be seen in Table 1.

Table 1 - examples of concepts related to robot structure.

Concept	Instance	Description
ServiceRobot	Bill	Bill is the service robot designed in our laboratory
Capability	Communication	This is a question/answer capability that the human ask a question, the robot process it and says the answer
ServiceRobotBehavior	VoiceCommunication	This is an implementation of the Capability <i>Communication</i> that uses voice to perform the communication. The human says something using voice and receives the answer hearing a sound.
PrimitiveSkill (Sensing)	Hear	A PrimitiveSkill that captures a sound and transforms it into text
PrimitiveSkill (Cognition)	FindAnswer	A PrimitiveSkill that receives a text as input (e.g., question), produces an answer (e.g., using LLM), returns a text as output (e.g., the question answer)
PrimitiveSkill (Actuating)	Speak	A PrimitiveSkill that receives a text as input, converts to a sound as output
DevicePart (RobotSensingPart)	Microphone_Lark_M2	A microphone sensor that captures a sound (e.g., a human voice when speaking to the robot)
SoftwarePart	Whisper_OpenAI	A software that processes the sound captured by a microphone, i.e., coverts this sound to text.

Contract	HearSensorContract	This contract links the PrimitiveSkill <i>Hear</i> with the DevicePart <i>Microphone_Lark_M2</i>
Requirement	The microphone should be able to capture sound from the distance of 1 metro from the operator	This requirement Indicates that the PrimitiveSkill <i>Hear</i> requires a microphone (DevicePart) with a specific feature

1.2 Concepts related to the Environment

ServiceRobots execute tasks in a **Scenario** where they can interact with **Elements**, such as person and objects. Figure 2 shows the metamodel defined to represent the environment where a **ServiceRobot** can operate.

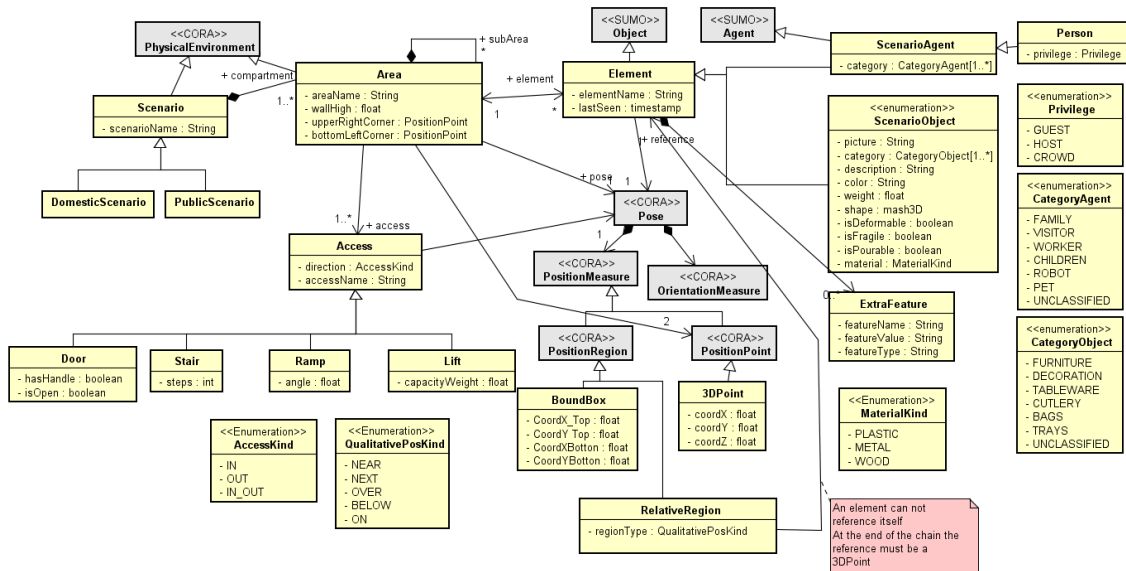


Figure 2: Environment definition metamodel.

Scenario: is a *PhysicalEnvironment* (from CORA) where a *ServiceRobot* performs tasks. It contains at least one *Area*. A *Scenario* can be of two types: *DomesticScenario* or *PublicScenario*.

DomesticScenario: is a *Scenario* in a typical domestic setting, e.g., an arena of the RoboCup@home, or a real-world house.

PublicScenario: is a non-structured place, such as, a restaurant, a hotel or a supermarket.

Area: is a *PhysicalEnvironment* (from CORA) that represents the parts of a *Scenario*. It may be composed of another *Area* (subarea) and has one or more *Access*. Examples of *Areas* in a *DomesticScenario* are bedroom, dining room, living room, and kitchen.

Examples of *Area* in a *PublicScenario* are reception, and rooms in a hotel. In this, the room may be composed of a bedroom and a bathroom (both are represented as *Area* in the ontology). Other examples of *Area* in a *PublicScenario* are the kitchen and the lounge of a restaurant. An *Area* has a name, a wallHeight (in meters), the coordinates of an upperRightCorner and a bottomLeftCorner forming a rectangle that represents the *Area*. Besides, an *Area* also has a *Pose*.

Access: is a means of entering an *Area*. An access may represent a connection between *Areas*. It has a name, a direction (*AccessKind*) and a *Pose*.

AccessKind: defines the direction of an *Access*: IN (only entrance) OUT (only exit) or IN_OUT (both entrance and exit).

Door: is a kind of *Access* that can be opened or closed. A *Door* may have a handle (hasHandle attribute) and has a state indicating if it is opened or closed (isOpen attribute)

hasOpeningDevice: when true, it indicates that there is a device (e.g., a handle or a button) that can be used to open the door; otherwise, there isn't such a device.

isOpen: when true, indicates that the door is open, otherwise the door is closed.

Stair: is a kind of *Access* that connects lower and higher levels using steps.

Ramp: is a slope or inclined plane for joining two different levels. Slope can be expressed in angles, gradients or grades.

Lift: is a kind of *Access* that is a transportation device like a box that moves up and down. It has a capacity indicating the maximum weight supported.

Element: extends *Object* ("normal physical objects" from SUMO) and is anything or anybody inside an *Area*. An *Element* has a *Pose* and the timestamp with the last time the pose was captured. Usually, the recognition probability is directly related to the amount of information that the service robot has about the element. *Elements* can be of two types: *ScenarioAgent* or *ScenarioObject*. Elements may be **recognized** or not by the robot according to their characteristics (attributes and extraFeature). Recognition comprises **Element classification**, i.e. the robot associates an element to a category according to its features; and / or **Element identification**, i.e. the robot identifies uniquely an instance using its features.

ExtraFeature: is any extra feature modeled to represent information of an *Element* that is not explicitly defined in the current version of the ontology. Examples of *ExtraFeature* for a *Person* is his cloth color. Examples of *ExtraFeature* for a *ScenarioObject* is its texture.

ScenarioAgent: is an *Agent* (from SUMO) "Something or someone that can act on its own and produce changes in the world". The **ServiceRobot** can classify a

ScenarioAgent according to one or more categories (**CategoryAgent**) and identify uniquely using extra features, e.g. biometric.

CategoryAgent: classifies a *ScenarioAgent* according to its type. When the **ServiceRobot** does not know a **ScenarioAgent** category it is set as Unclassified. Examples of **CategoryAgent** are:

- FAMILY
- VISITOR
- WORKER
- CHILDREN
- ROBOT
- PET
- UNCLASSIFIED

Person: specializes *ScenarioAgent* and represents a human being participating in the *Scenario*. A Person is associated with one or more Privileges (Privilege). For example a person from the family that has the privilege of HOST and a visitor that has the privilege of GUEST.

Privilege: defines a *ScenarioAgent*'s special rights, advantage, or immunity granted when interacting with the *ServiceRobot*. The *ScenarioAgent*'s rights are the command levels over the behavior or tasks the robot can perform. Examples of **Privilege** are:

- GUEST, interacts with the *ServiceRobot*, but does not have permission to operate it;
- HOST, has permission to give orders/commands to the robot;
- CROWD, does not interact with the robot;

ScenarioObject: specializes *Element* and represents things arranged in an *Area*. A *ScenarioObject* has a picture, can be classified according to one or more categories (**CategoryObject**) and identify uniquely using their features, e.g. picture. It has a description, picture, a color, weight (based on weight it can be categorized as heavy object, when weight is between 1.0kg and 1.5kg or light otherwise), shape (based on shape it can be categorized as tiny object, when it is no bigger than 5cm, or big otherwise), can be deformable (i.e. a flexible object that may appear in different shapes, e.g., cloth), fragile (i.e. an easy-to-break object, e.g., egg), and pourable (i.e. an object whose content can be poured, e.g., jug), and is made of a material (**MaterialKind**).

MaterialKind: is the physical substance that the object is made from, e.g., plastic, metal, wood.

CategoryObject: classifies a *ScenarioObject* according to its type. When a robot does not know a *ScenarioObject* category it is set as Unclassified. Examples of **CategoryObject** are

- Furniture, *ScenarioObject* that the ServiceRobot **does not intend** to move during a task. Examples of furniture in the RoboCup@home domain are bed, couch, table, chair, trash bins, television, cupboard, bookcase, coat rack, Dishwasher, Sink and Powered Refrigerator;
- Decoration, *ScenarioObject* used to decorate the area, e.g. a plant jar;
- Tableware, e.g. dish, bowl, cup, and napkin;
- Cutlery, e.g. fork, knife, and spoon;
- Bags, e.g. lightweight with stiff, vertical handles; and trash Bags, e.g. big plastic trash bags, preferably with handles
- Trays, e.g. Tray or basket, intended for two-handed manipulation;
- Unclassified, *ScenarioObjects* that the robot can not classify.

Pose (from CORA): “A position and an orientation constitute a pose. The pose of an object is the description of any position and orientation measurements of that object. “

Orientation Measure (from CORA): is essentially a measure (or observation) attributed to the orientation of a given (physical) object. In this context, orientation denotes the information regarding **where the object is pointing to** in relation to the reference object of the orientation coordinate system

Position Measure (from CORA): “A measure (SUMO) attributed to a (physical) object (SUMO) describing its position. A position can be described by a point or a region. For instance, one can describe a robot as positioned at coordinates (x, y) in the coordinate system, or at the front of the box, where ‘front’ comprises a conical region centered on the box and pointed forward.”

Position point (from CORA): “A point in a position coordinate system. It denotes a precise indication of position of a given object. Position points are always defined in a single coordinate system.”

3DPoint: is a 3D point in a position coordinate system indicating a specific location.

Position Region (from CORA): “An abstract region in a position coordinate system. More specifically, a position region is defined by position points in a given coordinate system. It defines qualitative positions such as ‘left of’, ‘in front of’, ‘on top of’, etc. These expressions define regions in relation to a reference object in which other objects are placed. A position region is always generated by a given spatial operator applied to a list of reference objects.”

BoundingBox: is a *PositionRegion* (CORA), i.e., a set of position points in a given coordinate system, delimited by a rectangle surrounding an *Element*.

RelativeRegion: is a *PositionRegion* (CORA) indicating a qualitative position about a referenced *Element*. Possible qualitative positions are predefined in the *QualitativePosKind*.

QualitativePosKind: represents a domain for possible relative positions. Examples are LEFT, RIGHT, ONTOP, BELOW.

Additional examples of the mentioned concepts can be seen in Table 2.

Table 2: Examples related to environment.

Concept	Example	Description
DomesticScenario	RoboCup@Home Arena	The RoboCup@home arena simulating a real apartment
Area	LivingRoom	The living room of the apartment
Area	Kitchen	The kitchen of the apartment
Access (Door)	FrontDoor	An access between the LivingRoom and the Kitchen
Person	Jhon	A person that interacts with the robot
ScenarioObject	Chair	A furniture (CategoryObject=FURNITURE) in the living room
ScenarioObject	Plate	An object (CategoryObject=TABLEWARE) in the kitchen

1.3 Concepts related to the Task

In order to serve humans, a **ServiceRobot** performs **Tasks**. Figure 3 shows the metamodel for task definition.

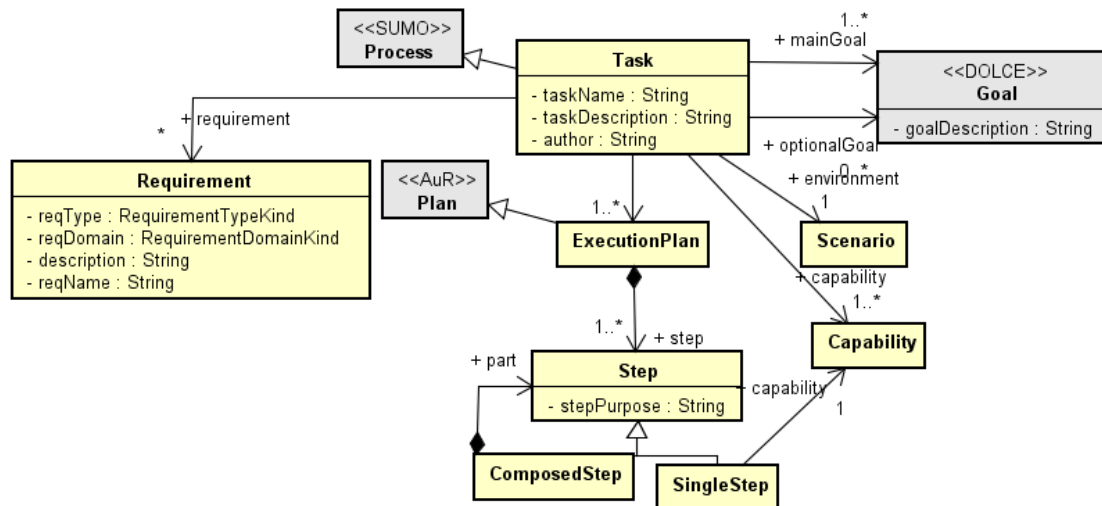


Figure 3: Task definition metamodel

Task: is a Process (from SUMO) that represents the work performed by the *ServiceRobot* when serving a human. A *Task* has a name, a description, main goals (at

least one), and may have optional goals. *Tasks* are performed in a set of steps organized as an *ExecutionPlan*.

Name defines the task name, e.g., Receptionist.

Description gives a brief explanation of the task, e.g., “The robot has to take new guests to the living room to introduce them and offer a free place to sit”.

Main goal: describes what the *serviceRobot* wants to achieve, e.g. “Introduce newcomers to a party and maintain appropriate gaze direction (at a person, the direction of navigation)”.

Optional goal: adds optional purposes, e.g., “Open the entrance door for each arriving guest” and “Describe the guest for the other people”.

ExecutionPlan: is a Plan (from AuR) with an arrangement of steps for *Task* execution. In this arrangement, Steps can be orchestrated in parallel or sequentially using, for example, a behavior tree or UML state machine.

Step: represents a stage of the *ExecutionPlan* with a particular purpose. There are two types of *Step*: *ComposedStep* and *SingleStep*.

ComposedStep: is a *Step* composed of other *Steps*. The composition of steps must end in a *SingleStep*.

SingleStep: is a *Step* that can not be composed of another *Step* (e.g. considering a tree, steps are the leaves of the tree). It refers to a *Capacity*.

Additional examples of the mentioned concepts can be seen in Table 3.

Table 3: examples of concepts related to task definition.

Concept	Example	Description
Task	Speech Recognition	A task that aims to evaluate the robot ability to recognize and answer five questions without asking for confirmation
Execution Plan	Represented in the following StateChart	A plan that orchestrates the steps towards the task definition
Step (SingleStep)	StartTask	This step represents the Capability (behavior) that a human says a password to call the robot attention, e.g. Hello robot, before starts a conversation
Step (SingleStep)	VoiceCommunication	This step represents the Capability (behavior) that a human have a conversation with the robot, e.g. the

		human asks a question, the robot processes it and says the answer
Step (SingleStep)	FinishTask	This step represents the Capability (behavior) that the robot announces the end of a task

Figure 4 presents an example of an execution plan for the task speech recognition, described on table 3.

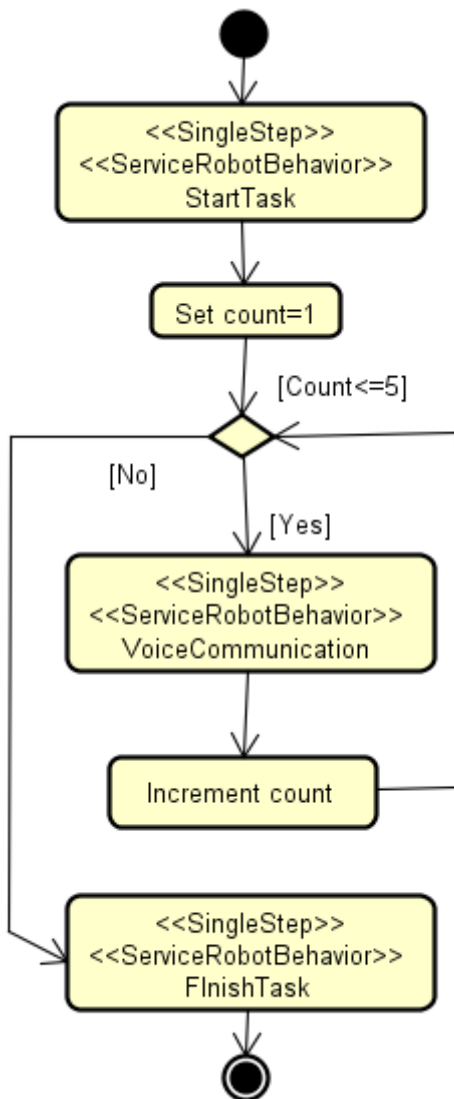


Figure 4: Execution plan of the Speech recognition task.

7. ROSIE implementation

ROSIE is implemented in Protegé tool and available at the following GitHub project

<https://github.com/anapatriciamagalhaes/ROSIE/tree/main>

Table 4 lists ROSIE's properties and axioms, according to the following legend:

Domain - domain name, applied to the property

Property - property name

Range - range name, applied to the property

F - Functional property

IF - Inverse Functional property

As - Asymmetric property

Ir - Irreflexive property

Opposite - name of the opposite property related to the property in col two

Table 4: Property details

Domain	Property	Range	F	IF	As	Ir	Opposit
ServiceRobot	hasComponent	ServiceRobotPart		x			isComponentOf
ServiceRobotPart	hasSubComponent	ServiceRobotPart		x	x	x	isSubcomponentOf
ServiceRobotPart	isParent	PartConnection					ConnectParent
ServiceRobotPart	isChild	PartConnection					ConnectChild
ServiceRobot	Performs	Capability					isPerformedBy
Capability	isImplementedAs	ServiceRobotBehavior		x			
ServiceRobotBehavior	Coordinates	PrimitiveSkill					
Contract	involvesPart	ServiceRobotPart					isPartOfContract
Contract	involvesAction	PrimitiveSkill					isActionOfContract
Requirement	hasDomain	RequirementDomainKind					
Requirement	hasType	RequirementDomainType	x				
ServiceRobotPart	requiresReqPart	Requirement					
PrimitiveSkill	RequiresReqAction	Requirement					
Task	RequiresReqTask	Requirement					

Scenario	isComposedOf	Area		x			isPartOf
Area	isDividedIn	Area		x		x	
Area	contains	Element		x			isContainedIn
Area	isAccessedBy	Access					
Access	isDirectedBy	AccessKind	x				
Area	isPositionedPArea	Pose	x	x			
Element	isPositionedPElem	Pose	x	x			posesElem
Access	isPositionedPAccess	Pose	x	x			
Element	isDescribedBy	ExtraFeature		x			
RelativeRegion	refersTo	Element	x				
RelativeRegion	isRelatedTo	QualitativePosKind	x				relates
Area	isMeasuredAs	PositionPoint					
Pose	hasPosition	PositionMeasure	x				positions
Pose	hasOrientation	OrientationMeasure	x				orients
ScenarioAgent	isCharacterizedAs	CategoryAgent					characterizes
Person	isGrantedBy	PrivilegeKind					grants
ScenarioObject	isCategorizedCatObject	CategoryObject					categorizesCatObject
ScenarioObject	isMadeOf	MaterialKind					
Task	hasGoal	Goal					
Task	isPlannedAs	ExecutionPlan		x			plans
ExecutionPlan	orchestrates	Step					isOrchestratedIn
SingleStep	represents	Capability	x				
CoposedStep	composes	Step					isStageOf
Task	hasOptionalGoal	Goal					
Task	isExecutedIn	Scenario					
Task	uses	Capability					

References

- IEEE Standard Ontologies for Robotics and Automation. IEEE Std 1872™-2015

- IEEE Standard for Autonomous Robotics (AuR) Ontology. IEEE Std 1872.2 TM -2021.
- ISO 8373 Robotics Vocabulary. Available at <https://cdn.standards.iteh.ai/samples/75539/1bc8409322eb4922bf680e15901852d2/ISO-8373-2021.pdf> accessed 23/04/2024.
- SUMO, The Suggested Upper Merged Ontology. Available at www.ontologyportal.org accessed 23/04/2024.
- SysML profile. Available at <https://www.omg.sysml.org/SysML-2.htm> accessed 23/04/2024.
- Vital, Luciane; Café, Ligia. Ontologias e Taxonomias: diferenças. Perspectivas em Ciência da Informação, v.16, n.2, p.115-130, abr./jun. 2011

Modifications:

18/08 - change the concept terminology from PrimitiveAction to PrimitiveSkill